

Từ prototype lên mass production (From Prototype to Mass Production)

PHỤ THUỘC C/14-hieu-chuan-nhu-mot-nhanh-cua-slam (hiệu chuẩn — ở đây nó trở thành một quy trình có chỉ tiêu thời gian và chất lượng); C/18-danh-gia-va-phuong-phap-luan (đánh giá — ở đây nó trở thành regression test và chỉ tiêu nghiệm thu); D/19-ben-vung-trong-the-gioi-thuc (chế độ hồng — ở đây thành fleet analytics); D/20-toi-uu-hieu-nang-va-he-thong (ràng buộc phần cứng — nên đọc trước chương này).

TÓM TẮT

Đây là chương duy nhất trong cây viết cho mục tiêu sản xuất hàng loạt, và nó là chương ít tài liệu nhất. Nội dung tổ chức theo bốn nhóm, tương ứng bốn nhóm quyết định mà một dự án phải đưa ra. *A — hiệu chuẩn ở quy mô sản xuất*: đồ gá tự động, chu trình dưới một phút, dung sai cơ khí, trôi theo thời gian, và một quyết định tốn nhiều tiền là hiệu chuẩn từng máy hay dùng tham số chung cho cả lô. *B — độ tin cậy*: định nghĩa chế độ hồng, giám sát sức khỏe tại hiện trường, watchdog, logging, và fleet analytics — vòng lặp biến hàng nghìn máy thành một công cụ nghiên cứu. *C — kiểm thử*: bộ dữ liệu nội bộ, regression tự động, hardware-in-the-loop, ma trận thử nghiệm, và nguyên tắc không bao giờ tin benchmark công khai là đủ. *D — ràng buộc sản phẩm*: chọn cấu hình cảm biến, chỉ phí BOM, nhiệt, điện, OTA, quyền riêng tư. Điều quan trọng nhất về chương này là *thứ tự*: phần lớn các quyết định ở nhóm A và D phải đưa ra *trước khi chốt thiết kế cơ khí*, khi chưa ai thấy chúng cần thiết — và không sửa được sau khi mở khuôn.

KIẾN THỨC CẦN CÓ TRƯỚC

- **đồ gá hiệu chuẩn (calibration fixture)** — thiết bị cơ khí giữ sản phẩm và target ở các tư thế đã biết, để quy trình hiệu chuẩn chạy tự động và lặp lại được trên dây chuyền.
- **golden sample** — một hoặc vài đơn vị sản phẩm được đo rất kỹ và giữ làm chuẩn tham chiếu, dùng để phát hiện trôi của chính quy trình sản xuất và quy trình đo.
- **SPC — statistical process control** — theo dõi thống kê của một tham số qua các lô sản xuất, để phát hiện khi quy trình bắt đầu trôi *trước khi* sản phẩm bị lỗi.
- **phân tích độ nhạy** — đo xem sai lệch của một tham số đầu vào (dung sai cơ khí, sai số hiệu chuẩn) gây ra sai lệch bao nhiêu ở đầu ra, để biết dung sai nào cần siết và dung sai nào nới được.
- **fleet analytics** — thu thập và phân tích dữ liệu vận hành từ toàn bộ đội thiết bị đang hoạt động, để tìm ra các chế độ hồng mà phòng thí nghiệm không tái hiện được.
- **HIL — hardware-in-the-loop** — chạy phần mềm trên phần cứng thật với dữ liệu được phát lại, để kiểm thử mà không cần môi trường thật.
- **MTBF — mean time between failures** — thời gian trung bình giữa hai lần hỏng. Với SLAM, cần định nghĩa rõ “hỏng” nghĩa là gì trước khi con số này có nghĩa.

CÂU HỎI MỞ ĐẦU

- Q1.** Bạn có một nguyên mẫu chạy tốt và cần sản xuất một nghìn máy. Nếu quyết định quan trọng nhất phải đưa ra, và nói vì sao nó phải đưa ra *trước* khi chốt thiết kế cơ khí.

- Q2. Hiệu chuẩn từng máy mất ba phút và tốn tiền; dùng tham số chung cho cả lô thì miễn phí. Nếu đại lượng quyết định lựa chọn ấy, và cách đo nó.
- Q3. Hệ thống của bạn chạy hoàn hảo trên EuRoC và trên bộ dữ liệu nội bộ. Nếu ba lý do vì sao nó vẫn có thể thất bại ở nhà khách hàng đầu tiên.
- Q4. Một nghìn máy đang chạy và bạn nhận được một trăm báo cáo lỗi mỗi tháng, phần lớn mơ hồ. Nếu thứ bạn cần thiết kế *từ trước* để biến các báo cáo ấy thành thông tin sửa được.
- Q5. Bạn phải chọn cấu hình cảm biến cho một robot dịch vụ trong nhà. Trình bày cách biến câu hỏi này thành một bài toán có tiêu chí, và nêu kết luận cùng điều kiện của nó.

1.1 Đặt vấn đề

Ba phần trước và hai chương trước dựng một hệ thống chạy được. Chương này về việc biến nó thành một nghìn hệ thống chạy được ở một nghìn nơi khác nhau, không có bạn ở đó.

Bài toán gốc: khoảng cách giữa *một máy chạy được* và *mọi máy chạy được* lớn hơn nhiều so với trực giác, và nó lớn theo những hướng mà kỹ thuật thuật toán không chạm tới.

Ba nguồn của khoảng cách ấy.

Một — biến thiên giữa các đơn vị. Nguyên mẫu của bạn có một camera cụ thể lắp ở một vị trí cụ thể. Một nghìn máy có một nghìn camera hơi khác nhau, lắp ở các vị trí hơi khác nhau trong dung sai cơ khí. Tham số hiệu chuẩn của máy này không dùng cho máy kia — hoặc có dùng được, và câu hỏi “có hay không” là một quyết định tốn nhiều tiền (mục 2C).

Hai — biến thiên giữa các môi trường. Bạn thử trong văn phòng của mình. Khách hàng dùng nó trong một nhà kho có sàn bóng, một căn hộ có nhiều gương, một nhà xưởng có bụi. Mỗi môi trường kích hoạt một mục khác trong danh sách ở D/19.

Ba — biến thiên theo thời gian. Nguyên mẫu chạy vài giờ. Sản phẩm chạy vài năm, trong đó ống kính bám bụi, keo lão hoá, môi trường thay đổi, và phần mềm được cập nhật.

Ai gặp bài toán này? Chỉ những người thật sự sản xuất, và đó là lý do tài liệu về nó gần như không tồn tại trong học thuật. Kiến thức thuộc về các công ty; nó không tổng quát hoá sạch sẽ thành định lý; và mỗi ràng buộc là riêng của một sản phẩm.

CẠM BẮY

Toàn bộ chương này là nguyên tắc và danh mục kiểm, không phải số liệu.

Tôi *không* có dữ liệu fleet thật, và mọi con số xuất hiện trong chương — “dưới sáu mươi giây”, “một nghìn máy”, các tỉ lệ phần trăm — là **ví dụ minh họa tự đặt** để lập luận có chỗ bám, không phải kết quả đo. Theo `learning-rules.md` mục 5, tôi phải nói rõ điều này một lần ở đầu chương thay vì rải cảnh báo khắp nơi.

Điều tôi đưa ra là *cấu trúc của các quyết định*: quyết định nào phải đưa ra trước quyết định nào, đại lượng nào quyết định mỗi lựa chọn, và cách đo đại lượng ấy. Cấu trúc ấy tôi tin là đúng và nó suy ra được từ các chương trước. Các con số thì phải do chính dự án của bạn đo.

Và một cảnh báo về cách đọc: chương này liệt kê rất nhiều việc phải làm. Không dự án nào làm hết ngay từ đầu. Cách dùng đúng là coi nó như một *danh mục để quyết định bỏ qua có ý thức* — biết rằng bạn đang bỏ qua điều gì và vì sao, thay vì phát hiện nó vào lúc đã quá muộn để sửa.

TRỰC GIÁC

Hình dung sự khác nhau giữa nấu một bữa ăn ngon và mở một nhà hàng.

Nấu một bữa ngon: bạn nêm, bạn chỉnh, bạn dùng đúng nguyên liệu mình thích. Kết quả phụ thuộc vào tay nghề của bạn ở thời điểm đó.

Mở nhà hàng thì khác về bản chất. Món ăn phải giống nhau ở mọi bàn, mọi ngày, do nhiều đầu bếp khác nhau nấu, với nguyên liệu mua từ nhiều nguồn khác nhau. Bạn không thể nêm từng đĩa.

Từ đó ra ba việc mà người nấu ăn ở nhà không bao giờ nghĩ tới, và chúng ánh xạ chính xác sang ba nhóm trong chương này.

Một — công thức phải đo được, không phải “nêm vừa ăn”. Bao nhiêu gram, bao nhiêu phút. Vì người nấu không phải bạn, và “vừa ăn” của họ khác của bạn. Đây là hiệu chuẩn và quy trình.

Hai — phải biết món nào bị hỏng mà không cần nêm. Nhiệt độ bếp, màu sắc, thời gian — các dấu hiệu gián tiếp thay cho việc nêm. Đây là giám sát sức khỏe.

Ba — phải có cách biết khách hàng nghĩ gì. Không phải mỗi lời phàn nàn lẻ tẻ, mà là: món nào bị trả lại nhiều nhất, vào giờ nào, ở bàn nào. Đây là fleet analytics, và nó là thứ biến một trăm lời phàn nàn mơ hồ thành một chẩn đoán.

Và có một việc thứ tư, quan trọng hơn cả ba và dễ bỏ quên nhất: *bạn phải quyết định kích thước bếp trước khi xây nhà hàng.* Sau khi đã xây, muốn thêm một cái lò nữa thì phải đập tường. Trong sản phẩm, những quyết định kiểu “đập tường” là dung sai cơ khí, chỗ đặt cảm biến, và loại camera — và chúng phải quyết vào lúc chưa ai thấy chúng quan trọng.

1.2 A – Hiệu chuẩn ở quy mô sản xuất

A. Phân tích độ nhạy phải chạy trước

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và là quyết định quan trọng nhất trong chương.

Câu hỏi: *dung sai cơ khí bao nhiêu là chấp nhận được?* Cụ thể: hai camera stereo lệch góc bao nhiêu độ thì vẫn đạt yêu cầu độ chính xác? Module camera xoay bao nhiêu so với thân máy?

Câu hỏi ấy *phải* được trả lời trước khi chốt thiết kế cơ khí, vì siết dung sai sau khi mở khuôn là bất khả về mặt kinh tế.

Quy trình trả lời, và nó là một quy trình chạy được chứ không phải một lời khuyên:

Bước một — xác định ngân sách sai số đầu ra. Từ yêu cầu sản phẩm: robot phải định vị chính xác bao nhiêu để làm được việc của nó.

Bước hai — lập danh sách các tham số đầu vào và độ tán dự kiến của chúng. Với mỗi tham số, một khoảng: tiêu cự lệch bao nhiêu giữa các đơn vị, góc lắp lệch bao nhiêu, đường cơ sở lệch bao nhiêu.

Bước ba — chạy Monte-Carlo. Nhiễu loạn từng tham số theo độ tán của nó trong mô phỏng, đo sai số đầu ra. Đây là C/18 mục 3B áp dụng cho một mục đích khác: mô phỏng để tách các nguồn.

Bước bốn — đọc kết quả theo hai chiều. Tham số nào mà sai lệch nhỏ gây sai số lớn thì *siết dung sai* hoặc *hiệu chuẩn từng máy*. Tham số nào mà sai lệch lớn cũng không sao thì *nới dung sai* để tiết kiệm chi phí cơ khí.

Chiều thứ hai là chiều hay bị bỏ quên và nó tiết kiệm tiền thật: phần lớn dự án siết mọi dung sai vì không biết cái nào quan trọng, và điều đó đắt một cách không cần thiết.

B. Quy trình hiệu chuẩn trên dây chuyền

Bốn ràng buộc mà quy trình phòng lab không có.

Thời gian. Mỗi giây trên dây chuyền là tiền. Một quy trình dùng Kalibr với người thao tác cầm bảng đi lại mất vài phút và không lặp lại được — không dùng được.

Tính lặp lại. Kết quả không được phụ thuộc vào người thao tác. Điều này đòi hỏi một *đồ gá*: một cơ cấu giữ sản phẩm và di chuyển target (hoặc ngược lại) qua một chuỗi tư thế cố định.

Chuỗi tư thế phải thoả điều kiện kích thích. Đây là chỗ A/05 Bảng kích thích và C/14 trả công trực tiếp. Chuỗi tư thế của đồ gá phải được *thiết kế* để mỗi tham số cần ước lượng đều được kích thích: đa dạng độ sâu cho tiêu cự, target ở bốn góc ảnh cho điểm chính và méo, xoay quanh nhiều trục nếu có hiệu chuẩn camera-IMU.

Điều đáng nhấn mạnh: chuỗi ấy nên được *kiểm bằng phân tích observability* trước khi chế tạo đồ gá — dựng bài toán MAP với chuỗi tư thế dự kiến và xem hiệp phương sai của các tham số. Nếu một tham số có hiệp phương sai lớn thì chuỗi chưa đủ, và phát hiện điều đó trên máy tính rẻ hơn nhiều so với phát hiện sau khi đã chế tạo đồ gá.

Tiêu chí đạt/không đạt tự động. Không có kỹ sư nhìn từng kết quả. Cần một bộ tiêu chí chạy tự động, và theo A/02 mục 4 thì RMS *không* phải tiêu chí đúng. Bốn tiêu chí đúng: độ phủ của target trên khung hình (kiểm được *trước* khi tối ưu — rẻ nhất, nên chạy đầu tiên); hiệp phương sai của từng tham số; hoa văn phần dư; và so với phân bố của lô (mục 2D).

C. Hiệu chuẩn từng máy hay dùng chung

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và là một quyết định có ảnh hưởng lớn tới chi phí.

Đại lượng quyết định là một *tỉ số*:

$$\frac{\text{độ tán của tham số trong lô}}{\text{mức tham số đó được phép sai theo ngân sách}}.$$

Từ số đo bằng cách hiệu chuẩn một mẫu vài chục máy và tính độ lệch chuẩn. Mẫu số đến từ phân tích độ nhảy ở mục 2A.

Nếu tỉ số nhỏ hơn một, tham số chung dùng được và bạn tiết kiệm được thời gian dây chuyền. Nếu lớn hơn một, phải hiệu chuẩn từng máy — hoặc siết dung sai cơ khí, vốn có thể rẻ hơn tùy trường hợp.

Và quyết định không nhất thiết đồng nhất cho mọi tham số. Thường gặp: *tiêu cự và méo* khá đồng đều trong một lô module cùng nguồn nên dùng chung được; *ngoại* — vị trí tương đối giữa các camera — phụ thuộc vào lắp ráp nên độ tán lớn hơn nhiều và thường phải đo từng máy. Một quy trình hỗn hợp — dùng chung nội, đo riêng ngoại — thường là điểm tối ưu, và nó nhanh hơn nhiều so với đo tất cả.

D. Theo dõi qua thời gian: golden sample và SPC

Hai cơ chế cần thiết và hay bị bỏ.

Golden sample. Giữ vài đơn vị đã được đo rất kỹ bằng một phương pháp độc lập và chính xác hơn. Chạy chúng qua dây chuyền định kỳ. Nếu kết quả trôi, thì *quy trình đo* đang trôi — không phải sản phẩm. Không có cơ chế này thì bạn không phân biệt được “lô này kém” với “máy đo bị lệch”.

SPC trên tham số hiệu chuẩn. Vẽ phân bố của từng tham số qua các lô. Một dịch chuyển của trung bình hoặc một giãn nở của độ tán là dấu hiệu quy trình sản xuất đang trôi — *trước khi* sản phẩm bị lỗi. Đây là công cụ chuẩn của sản xuất công nghiệp, và nó áp dụng trực tiếp cho tham số hiệu chuẩn mà ít ai làm.

Trôi sau khi xuất xưởng là chuyện của C/14 mục 4B và D/20 mục 5: nhiệt độ, rung, va đập, lão hoá keo. Hai cơ chế xử lý: hiệu chuẩn trực tuyến có cổng observability, và phát hiện để yêu cầu hiệu chuẩn lại.

1.3 B – Độ tin cậy và fleet analytics

A. Định nghĩa “hỏng” trước khi đo MTBF

MTBF là một con số vô nghĩa cho tới khi “hỏng” được định nghĩa. Với SLAM, cần một thang chứ không phải một ngưỡng, và nó nên khớp với thang chế độ suy giảm ở D/19 mục 6C.

Một thang hợp lý, từ nhẹ tới nặng: suy giảm có phát hiện (hệ chuyển chế độ và báo — không tính là hỏng); mất bám tạm thời có tự phục hồi; mất bám cần khởi động lại phần mềm; sai vị trí im lặng (ngghiêm trọng nhất, vì hậu quả không giới hạn); và bản đồ bị phá (mất dữ liệu).

Mỗi mức có một tỉ lệ chấp nhận khác nhau, và *sai vị trí im lặng* nên có tỉ lệ chấp nhận thấp hơn nhiều bậc so với các mức khác — đúng lập luận ở D/19 mục 6A.

B. Giám sát sức khoẻ tại hiện trường

Các đại lượng đã được liệt kê rải rác: C/18 mục 3C, D/19 khối *Cần nhớ*, D/20 mục 2. Ở đây là ba nguyên tắc để biến chúng thành một hệ thống.

Một — đo nguyên nhân, không chỉ hậu quả (D/19). Mật độ đặc trưng theo vùng ảnh nói nhiều hơn ATE.

Hai — hiệu chuẩn quan hệ giữa đại lượng thay thế và sai số thật, một lần, trong lab. Các đại lượng giám sát không đo sai số; chúng đo các điều kiện mà sai số nhỏ phụ thuộc vào. Quan hệ giữa chúng phải được thiết lập trên dữ liệu có chân trị rồi mang ra dùng. Bước này bị bỏ qua gần như mọi nơi, và không có nó thì các ngưỡng cảnh báo là số đoán.

Ba — ngân sách phải nằm trong ngân sách. Theo D/20, mọi thứ chạy mỗi khung đều tốn. Phần lớn đại lượng giám sát rẻ (vài chục con số mỗi khung, tổng hợp thành thống kê mỗi giây), nhưng vài cái không — tính phổ giá trị riêng của Λ chẳng hạn. Chạy cái đắt theo chu kỳ thưa hơn.

C. Logging: đủ để tái hiện, không nhiều hơn

Đây là một phần của câu trả lời cho câu hỏi mở đầu thứ tư.

Yêu cầu mâu thuẫn: log phải đủ để tái hiện một lỗi, nhưng không được tốn băng thông, bộ nhớ, hay quyền riêng tư.

Cách giải quyết là *ba tầng*.

Tầng một — luôn bật, rất rẻ. Các đại lượng giám sát tổng hợp theo giây, kèm nhiệt độ và trạng thái chế độ. Vài trăm byte mỗi giây. Đây là thứ cho phép fleet analytics.

Tầng hai — vòng đệm quay vòng, giữ vài chục giây gần nhất. Đủ chi tiết để tái hiện: mốc thời gian, quyết định keyframe, kết quả các cổng. Chỉ được *ghi ra* khi có sự kiện đáng chú ý — một lần mất bám, một cảnh báo. Đây là cơ chế quan trọng nhất, vì nó cho bạn dữ liệu về *chính khoảnh khắc* trước khi hỏng.

Tầng ba — ghi ảnh, chỉ khi được phép. Đắt và nhạy cảm về quyền riêng tư. Chỉ bật theo yêu cầu và có sự đồng ý.

Và theo D/20 mục 3B: log tầng hai chỉ có giá trị nếu hệ thống *tất định*. Nếu không, bạn có dữ liệu mà không tái hiện được.

D. Fleet analytics: vòng lặp quan trọng nhất

Đây là phần còn lại của câu trả lời cho câu hỏi mở đầu thứ tư, và tôi cho là đóng góp lớn nhất của chương.

Một nghìn máy chạy tám tiếng mỗi ngày là tám nghìn giờ dữ liệu mỗi ngày — nhiều hơn bất kỳ bộ dữ liệu nghiên cứu nào nhiều bậc. Nếu bạn thu được các đại lượng tổng hợp từ chúng, bạn có một công cụ mà không phòng thí nghiệm nào có.

Vòng lặp, bốn bước.

Một — thu thập các đại lượng tầng một từ toàn đội, cộng các bản ghi tầng hai khi có sự kiện.

Hai — phân cụm các sự kiện theo dấu hiệu, không theo mô tả của khách hàng. Các lần mất bảm có cùng hồ sơ đại lượng giám sát — cùng kiểu sụt đặc trưng, cùng kiểu tăng phần dư — nhiều khả năng có cùng nguyên nhân, dù khách hàng mô tả chúng bằng những từ khác nhau.

Ba — xếp hạng các cụm theo tần suất nhân mức nghiêm trọng. Đây là danh sách việc cần làm, và nó khác rất nhiều so với danh sách mà kỹ sư tự nghĩ ra.

Bốn — tái hiện cụm lớn nhất trong lab bằng dữ liệu tầng hai, sửa, và đo lại trên fleet ở phiên bản sau.

Điều quan trọng nhất về vòng lặp này: nó biến “một trăm báo cáo mơ hồ” thành “ba nguyên nhân xếp theo mức quan trọng”. Và nó chỉ khởi động được nếu ba thứ đã được thiết kế *từ trước*: các đại lượng giám sát, cơ chế log ba tầng, và tính tất định. Không thứ nào thêm vào được sau một cách rẻ.

Đây cũng là chỗ D/19 mục 6A trả công: một hệ biết mình sai tạo ra dữ liệu; một hệ im lặng thì không. Toàn bộ giá trị của fleet analytics phụ thuộc vào việc hệ thống phát hiện được thất bại của chính nó.

1.4 C – Kiểm thử

A. Bộ dữ liệu nội bộ

Đây là câu trả lời cho câu hỏi mở đầu thứ ba.

Ba lý do một hệ chạy hoàn hảo trên benchmark vẫn thất bại ở nhà khách hàng đầu tiên.

Một — benchmark không chứa các chế độ hỏng ở D/19. Chúng được thu trong điều kiện kiểm soát, không có gương, ít người đi lại, không có bụi ống kính.

Hai — benchmark quá ngắn. Không bộ dữ liệu chuẩn nào dài tám tiếng, nên các vấn đề ở C/13 mục 4 — kích thước bản đồ, lão hoá — không xuất hiện.

Ba — benchmark đo sai thứ. Chúng đo ATE trung bình trên các chuỗi hệ thống chạy được; sản phẩm cần tỉ lệ thất bại và hành vi ở đuôi (C/18 mục 5D).

Kết luận, và nó là một nguyên tắc cứng: *không bao giờ tin benchmark công khai là đủ.* Cần một bộ dữ liệu nội bộ phản ánh môi trường khách hàng, và nó nên gồm ba phần: các chuỗi *đại diện* (môi trường điển hình, dài), các chuỗi *ngịch cảnh* (bộ ở D/19 mini project), và các chuỗi *từ hiện trường* (dữ liệu tầng hai từ các lần hỏng thật — đây là phần có giá trị nhất và nó chỉ có được nhờ vòng lặp ở mục 3D).

B. Regression test tự động

Mỗi commit chạy toàn bộ bộ dữ liệu nội bộ và so với đường cơ sở. Ba chi tiết quyết định nó có dùng được hay không.

Một — so với phân vị, không chỉ trung bình (C/18, D/20). Một thay đổi làm giảm ATE trung bình mà tăng tỉ lệ thất bại là một hồi quy.

Hai — chạy nhiều *seed* nếu hệ không tắt định; nếu tắt định thì một lần là đủ, và đó là một lý do nữa để đầu tư vào tính tắt định.

Ba — nhiều chỉ tiêu, không một. ATE, tỉ lệ thất bại, tỉ lệ phát hiện thất bại, phân vị thời gian, bộ nhớ. Một bảng chứ không một con số.

C. HIL và ma trận thử nghiệm

HIL: chạy phần mềm trên phần cứng thật với dữ liệu phát lại. Nó bắt được các vấn đề mà chạy trên máy tính bỏ sót: thermal throttling (D/20 mục 5), giới hạn bộ nhớ, hành vi của ISP. Điều kiện để có giá trị: phát lại phải *đúng thời gian thực*, vì phần lớn vấn đề ở đây là vấn đề thời gian.

Ma trận thử nghiệm: liệt kê các chiều biến thiên — ánh sáng, loại sàn, tốc độ, tải, mức độ động của môi trường — và thử các tổ hợp. Không thể thử hết, nên chọn theo hai tiêu chí: các tổ hợp có khả năng xảy ra ở khách hàng, và các tổ hợp mà lý thuyết nói là khó (từ A/05 và D/19).

Điều đáng nhớ: ma trận nên được xây từ *hai nguồn* — kinh nghiệm về khách hàng, và lý thuyết về chỗ hệ thống suy biến. Chỉ dùng nguồn thứ nhất thì bỏ sót các trường hợp chưa gặp; chỉ dùng nguồn thứ hai thì thử nhiều thứ không bao giờ xảy ra.

I 1.5 D — Ràng buộc sản phẩm

A. Chọn cấu hình cảm biến

Đây là câu trả lời cho câu hỏi mở đầu thứ năm.

Cách biến nó thành bài toán có tiêu chí, ba bước, dùng lại các công cụ đã có.

Bước một — liệt kê *gauge cần phá*. Từ A/05: mono thuần thị giác có bảy bậc không quan sát được. Ứng dụng cần bao nhiêu trong số đó? Robot dịch vụ cần tỉ lệ metric (để đo khoảng cách thật) và cần hướng ổn định; nó *không* cần vị trí tuyệt đối toàn cục.

Bước hai — với mỗi ứng viên cảm biến, xem nó phá *gauge* nào và đòi điều kiện gì (C/15 Bảng cảm biến).

Bước ba — kiểm tính độc lập của chế độ hỏng (D/19). Hai cảm biến cùng chết trong bóng tối không cho khả năng phục hồi.

Áp dụng cho robot dịch vụ trong nhà, kết luận của tôi: *stereo + IMU + odometry bánh xe*, với *global shutter*.

Lập luận: stereo phá gauge tỉ lệ ngay từ khung đầu, làm khởi tạo trở nên tầm thường và xoá cả một lớp chế độ hỏng (B/10 mục 3A); IMU ghim roll với pitch, cho chuyển động tần số cao, và *quan trọng nhất* là nó sống khi thị giác chết; odometry bánh xe cho tỉ lệ độc lập và ràng buộc nonholonomic miễn phí, và nó sống khi cả thị giác lẫn IMU đều kém (C/15 mục 4A). Ba cảm biến có ba chế độ hỏng gần như độc lập: thị giác chết trong tối, IMU trôi khi đứng yên lâu, odometry nói dối khi trượt — và chúng kiểm chéo lẫn nhau.

Global shutter: theo A/02 mục 5A, nó xoá bỏ một lớp mô hình, một lớp tham số phải hiệu chuẩn, và một lớp chế độ hỏng chỉ xuất hiện khi rung. Chênh lệch giá vài độ la mỗi máy đổi lấy điều đó là một trong số ít quyết định trong cây mà phần cứng thắng phần mềm rõ ràng.

Điều kiện của kết luận này: nó dành cho robot mặt đất, trong nhà, hoạt động không giám sát, với ngân sách cho phép ba cảm biến. Nó *không* áp dụng cho drone (không có bánh xe), cho thiết bị cầm tay (không có odometry), hay cho sản phẩm cực rẻ (ngân sách chỉ đủ một camera). (*Đây là*

suy luận của tôi từ các chương trước, không phải kết quả của một khảo sát thị trường hay một thử nghiệm so sánh.)

B. OTA, phiên bản, và quyền riêng tư

Ba ràng buộc còn lại, mỗi cái một đoạn.

Cập nhật OTA cho phần mềm, bản đồ, và mô hình. Ba yêu cầu: định dạng bản đồ phải có phiên bản (C/13 mục 5A) — thêm sau rất đau; phải có cơ chế quay lui khi bản cập nhật gây hồi quy; và với mô hình học được, mỗi lần đổi mô hình đòi *tái kiểm định toàn bộ* (C/17), nên chi phí cập nhật cao hơn nhiều so với đổi một ngưỡng.

Quyền riêng tư. Theo C/13 mục 5B, bản đồ gồm điểm và descriptor là dữ liệu riêng tư — tái tạo được ảnh từ nó [1]. Ba hệ quả: tải bản đồ lên đám mây là truyền dữ liệu cá nhân; lưu trên thiết bị thì nó là mục tiêu; và log tầng ba (ảnh) phải có sự đồng ý rõ ràng. Điều tối thiểu: coi bản đồ là dữ liệu cá nhân trong mọi quyết định thiết kế, và nói rõ điều đó với khách hàng.

Nhiệt và điện. Theo D/20 mục 5: chọn SoC theo ngân sách nhiệt liên tục chứ không theo hiệu năng đỉnh, và *đặt một cảm biến nhiệt gần module camera*. Chi phí vài xu, giá trị chẩn đoán cao, và không gắn thêm được sau khi mở khuôn.

CẦN NHỚ

Thứ tự quan trọng hơn nội dung. Chương này liệt kê nhiều việc; điều quyết định là việc nào phải làm *trước*.

Bốn quyết định **không sửa được sau khi chốt cơ khí**, xếp theo mức tổn kém nếu sai:

Một — dung sai cơ khí, quyết bởi phân tích độ nhạy ở mục 2A. Sai thì hoặc phải mở khuôn lại, hoặc phải hiệu chuẩn từng máy suốt đời sản phẩm.

Hai — loại cảm biến và loại shutter. Global shutter và cấu hình cảm biến quyết định cả một lớp chế độ hồng có tồn tại hay không.

Ba — các cảm biến phụ trợ rẻ tiền: cảm biến nhiệt gần camera, encoder bánh xe. Vài xu mỗi cái, và chúng là thứ phân biệt được các nguyên nhân suy giảm.

Bốn — truy cập ảnh gần raw và điều khiển ISP, vốn là một yêu cầu với nhà cung cấp module phải nêu *trước khi chọn module*.

Và ba quyết định **phần mềm** nên đưa ra từ đầu vì thêm sau rất đắt: *tính tất định* (D/20 mục 3B); *định dạng bản đồ có phiên bản*; và *cơ chế giám sát cùng log ba tầng*, vì không có chúng thì vòng lặp fleet analytics ở mục 3D không khởi động được.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Kalibr [2]	Hiệu chuẩn đa cảm biến; quy trình thu dữ liệu	Điểm khởi đầu cho mục 2B — nhưng quy trình dây chuyền phải tự thiết kế, vì Kalibr không nhắm tới dây chuyền
maplab [3]	Nhiều phiên, quản lý bản đồ dài hạn	Hiếm framework nghĩ về hoạt động nhiều phiên; liên quan mục 5B
OpenLORIS [4]	Bộ dữ liệu nhiều phiên trong môi trường thật có thay đổi	Gần nhất với tinh thần mục 4A trong các bộ dữ liệu công khai
evo [5], rpg_trajectory_evaluation [6]	Công cụ đánh giá quỹ đạo	Nền cho regression test ở mục 4B; nhớ chọn đúng chế độ căn chỉnh

MINI PROJECT SLAM · Chạy phân tích độ nhạy cho sản phẩm của bạn, trước khi chốt cơ khí · 4 ngày

Mục tiêu	biến câu hỏi “dung sai bao nhiêu là đủ” từ một cuộc tranh luận thành một bảng số — và làm điều đó vào lúc còn sửa được.
Dữ liệu	không cần phần cứng: toàn bộ bài này chạy trong mô phỏng, và đó chính là điểm mạnh của nó.
Các bước	viết ra ngân sách sai số đầu ra từ yêu cầu sản phẩm; lập danh sách mười tham số đầu vào với độ tán dự kiến (nếu chưa biết, dùng dung sai mà nhà cung cấp cơ khí đề xuất làm điểm khởi đầu); chạy Monte-Carlo nhiều lần từng tham số riêng lẻ rồi nhiều lần tất cả cùng lúc; xếp hạng theo độ nhạy; với ba tham số nhạy nhất, tính tỉ số ở mục 2C để quyết định siết dung sai, hiệu chuẩn từng máy, hay cả hai; cuối cùng thiết kế chuỗi tư thế cho đồ gá và kiểm nó bằng phân tích observability theo mục 2B trước khi vẽ cơ khí.
Xong khi	bạn có một bảng mười tham số với ba cột: độ nhạy, độ tán dự kiến, và quyết định (nói / siết / hiệu chuẩn từng máy). Và bạn có một chuỗi tư thế đồ gá đã được kiểm rằng nó kích thích đủ mọi tham số cần ước lượng. Hai sản phẩm ấy là hai thứ đắt nhất để làm lại nếu bỏ qua, và chúng tốn bốn ngày trên máy tính.
Nối với	C/14-hieu-chuan-nhu-mot-nhanh-cua-slam cho phần lý thuyết của việc kiểm chuỗi tư thế; C/18-danh-gia-va-phuong-phap-luan cho phương pháp Monte-Carlo có kỷ luật.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Quyết định quan trọng nhất là *dung sai cơ khí*, và nó được trả lời bằng phân tích độ nhạy: nhiều loạn từng tham số trong mô phỏng theo độ tán dự kiến, đo sai số đầu ra, rồi siết dung sai cho các tham số nhạy và nói cho các tham số không nhạy. Nó phải đưa ra trước khi chốt cơ khí vì siết dung sai sau khi mở khuôn là bất khả về mặt kinh tế — lựa chọn còn lại là hiệu chuẩn từng máy suốt đời sản phẩm, vốn đắt hơn nhiều. Chiều *nói* cũng quan trọng và hay bị bỏ: phần lớn dự án siết mọi dung sai vì không biết cái nào quan trọng, và điều đó đắt một cách không cần thiết.
- A2. Đại lượng quyết định là tỉ số (*độ tán của tham số trong lô*) / (*mức tham số được phép sai theo ngân sách*). Từ số đo bằng cách hiệu chuẩn một mẫu vài chục máy và tính độ lệch chuẩn; mẫu số đến từ phân tích độ nhạy. Tỉ số nhỏ hơn một thì dùng chung được. Quyết định không cần đồng nhất cho mọi tham số: thường thì tiêu cự và méo khá đồng đều trong một lô module cùng nguồn nên dùng chung được, còn *ngoại* phụ thuộc lắp ráp nên độ tán lớn hơn và phải đo từng máy. Quy trình hỗn hợp — dùng chung nội, đo riêng ngoại — thường là điểm tối ưu và nhanh hơn nhiều so với đo tất cả.
- A3. (1) Benchmark *không chứa các chế độ hồng* ở D/19 — không có gương, ít người đi lại, không có bụi ống kính, vì chúng được thu trong điều kiện kiểm soát. (2) Benchmark *quá ngắn* — không bộ nào dài tám tiếng, nên các vấn đề về kích thước bản đồ và lão hoá ở C/13 không xuất hiện. (3) Benchmark *đo sai thứ* — chúng đo ATE trung bình trên các chuỗi hệ thống chạy được, trong khi sản phẩm cần tỉ lệ thất bại và hành vi ở đuôi. Kết luận: không bao giờ tin benchmark công khai là đủ; cần bộ dữ liệu nội bộ gồm chuỗi đại diện, chuỗi nghịch cảnh, và — có giá trị nhất — chuỗi từ hiện trường.
- A4. Ba thứ phải thiết kế từ trước: (1) *các đại lượng giám sát* đo nguyên nhân chứ không chỉ hậu quả; (2) *cơ chế log ba tầng* — tầng một luôn bật và rẻ, tầng hai là vòng đệm quay vòng chỉ ghi ra khi có sự kiện (đây là tầng quan trọng nhất vì nó cho dữ liệu về chính khoảnh khắc trước khi hỏng), tầng ba là ảnh và chỉ khi được phép; (3) *tính tất định*, vì không có nó thì log tầng hai vô dụng. Có ba thứ đó thì vòng lặp fleet analytics chạy được: phân cụm sự kiện *theo dấu hiệu* chứ không theo mô tả của khách hàng, xếp hạng cụm theo tần suất nhân mức nghiêm trọng, rồi tái hiện cụm lớn nhất trong lab. Nó biến một trăm báo cáo mơ hồ thành ba nguyên nhân xếp hạng.
- A5. Ba bước: liệt kê gauge cần phá (A/05); với mỗi ứng viên xem nó phá gauge nào và đòi điều kiện gì (C/15); kiểm tính *độc lập của chế độ hồng* (D/19). Với robot dịch vụ trong nhà, kết luận của tôi là *stereo + IMU + odometry bánh xe, với global shutter*: stereo phá gauge tỉ lệ ngay từ khung đầu và xoá cả một lớp chế độ hồng khởi tạo; IMU ghim roll-pitch và sống khi thị giác chết; odometry cho tỉ lệ độc lập, ràng buộc nonholonomic miễn phí, và sống khi cả hai kia kém. Ba chế độ hồng gần như độc lập nên chúng kiểm chéo lẫn nhau. Điều kiện: kết luận này dành cho robot mặt

đất trong nhà, hoạt động không giám sát, có ngân sách cho ba cảm biến — nó không áp dụng cho drone, thiết bị cầm tay, hay sản phẩm cực rẻ.

NÓI LẠI THẬT ĐƠN GIẢN

Nấu một bữa ăn ngon và mở một nhà hàng là hai việc khác nhau về bản chất, không chỉ về quy mô. Nấu ở nhà: bạn nêm, bạn chỉnh, bạn dùng nguyên liệu mình thích. Kết quả phụ thuộc vào tay nghề của bạn hôm đó.

Mở nhà hàng: món phải giống nhau ở mọi bàn, mọi ngày, do nhiều người nấu, với nguyên liệu từ nhiều nguồn. Bạn không thể nêm từng đĩa.

Từ đó ra bốn việc mà người nấu ăn ở nhà không bao giờ nghĩ tới.

Công thức phải đo được, không phải “nêm vừa ăn” — vì người nấu không phải bạn và “vừa ăn” của họ khác của bạn.

Phải biết món nào hồng mà không cần nêm — nhìn màu, đo nhiệt độ bếp, canh thời gian. Các dấu hiệu gián tiếp thay cho việc nêm.

Phải biết khách nghĩ gì một cách có hệ thống — không phải từng lời phàn nàn lẻ tẻ mà là: món nào bị trả lại nhiều nhất, vào giờ nào, ở bàn nào. Một trăm lời phàn nàn mơ hồ gộp lại có thể chỉ ra đúng một cái bếp bị lệch nhiệt.

Và việc thứ tư, quan trọng nhất và dễ quên nhất: *phải quyết kích thước bếp trước khi xây nhà hàng*. Sau khi xây xong, muốn thêm một cái lò thì phải đập tường.

Trong một sản phẩm có camera, những quyết định kiểu “đập tường” là: hai camera cách nhau bao xa và được phép lệch bao nhiêu độ; dùng loại cảm biến ảnh nào; có gắn một cái đo nhiệt độ bên cạnh ống kính hay không. Ba thứ ấy phải quyết vào lúc chưa ai thấy chúng quan trọng — và đó chính là lý do chúng hay bị bỏ qua.

Cách duy nhất biết trước là *thử trên máy tính*: giả sử camera lệch nửa độ thì kết quả sai bao nhiêu? Nếu sai nhiều thì phải làm cơ khí chính xác hơn, hoặc phải đo riêng từng máy. Nếu sai ít thì có thể làm lỏng tay và tiết kiệm. Bốn ngày ngồi máy tính đổi lấy việc không phải mở lại khuôn nhựa — đó là phép tính dễ nhất trong cả chương.

Chỗ không rút gọn được: chỗ không rút gọn được là việc *phải quyết định khi chưa có dữ liệu*. Bạn cần biết độ tán của tham số trong lô để quyết dung sai, nhưng bạn chỉ đo được độ tán sau khi đã sản xuất một lô — tức sau khi đã chốt dung sai. Cách duy nhất phá vòng là làm một lô mẫu nhỏ trước, và đó là một khoản chi phải thuyết phục được người trả tiền vào lúc chưa ai thấy nó cần.

Một câu: *Khoảng cách từ một máy chạy được tới nghìn máy chạy được nằm ở những quyết định phải đưa ra trước khi ai thấy chúng quan trọng.*

1.6 Điều gì chứng minh cách hiểu này sai

Toàn bộ chương này là *cấu trúc của các quyết định* chứ không phải các sự kiện đo được, nên nó bị bác bỏ theo một cách khác với các chương trước: không phải bằng một phép đo mà bằng một kinh nghiệm triển khai mâu thuẫn.

Mệnh đề chính — *phân tích độ nhạy phải chạy trước khi chốt cơ khí* — bị bác bỏ nếu trong thực tế, việc hiệu chuẩn từng máy rẻ tới mức không đáng siết dung sai cho bất kỳ tham số nào. Điều đó *có thể* đúng với sản phẩm sản lượng thấp giá cao, nơi ba phút dây chuyền không đáng kể. Nên phát biểu chặt hơn: mệnh đề đúng khi thời gian dây chuyền là ràng buộc, và mất hiệu lực khi không.

Mệnh đề thứ hai — *stereo + IMU + odometry là điểm ngọt cho robot mặt đất* — là suy luận của tôi từ ba chương trước, *không phải kết quả của một khảo sát hay một thử nghiệm so sánh*. Nó bị bác bỏ nếu một sản phẩm thành công trên thị trường dùng cấu hình rẻ hơn — một camera mono cộng IMU chẳng hạn — với tỉ lệ hỏng chấp nhận được. Tôi ngờ rằng điều này *có* xảy ra ở phân khúc giá rẻ, và nếu vậy thì kết luận đúng là “điểm ngọt phụ thuộc vào tỉ lệ hỏng mà thị trường chấp nhận”, không phải “ba cảm biến là câu trả lời”. Đây là mệnh đề tôi ít chắc nhất trong chương.

Mệnh đề thứ ba — *fleet analytics là vòng lặp quan trọng nhất* — bị bác bỏ nếu các chế độ hồng thật đủ đơn giản để tái hiện trong lab mà không cần dữ liệu hiện trường. Với một sản phẩm hoạt động trong môi trường kiểm soát — một nhà máy có điều kiện ổn định — điều đó có thể đúng. Với sản phẩm tiêu dùng thì tôi cho là không.

Và một cảnh báo bao trùm: chương này được viết *không có kinh nghiệm vận hành fleet thật*. Nó suy ra từ nguyên lý và từ các chương trước. Ai đã vận hành một đội thiết bị thật sẽ biết những thứ không có ở đây, và nếu bạn trở thành người đó thì hãy sửa chương này bằng kinh nghiệm của mình.

1.7 Kết nối

Chương này là chỗ hội tụ của gần như mọi chương trong cây, và cách đọc đúng là đọc nó như một bản đồ ngược.

Nối lên trên theo từng mục. Mục 2 (hiệu chuẩn) lấy từ A/02 (mô hình và chẩn đoán), A/05 (điều kiện kích thích), A/06 (hiệp phương sai tham số), và C/14 (khung hợp nhất) — và biến chúng thành một quy trình có chỉ tiêu. Mục 3 (độ tin cậy) lấy từ D/19 (chế độ hồng và phát hiện) và D/20 (logging và tính tắt định). Mục 4 (kiểm thử) lấy từ C/18 (phương pháp đánh giá) và D/19 (bộ nghịch cảnh). Mục 5 (ràng buộc sản phẩm) lấy từ A/05 và C/15 (chọn cảm biến theo gauge), A/02 (global shutter), C/13 (quyền riêng tư và định dạng bản đồ), D/20 (nhiệt).

Nối xuống dưới: D/22-huong-nghien-cuu-con-trong là chương cuối, và ba trong tám hướng ở đó — hiệu chuẩn có ý thức observability, dự đoán thất bại, hệ tự chữa — đều mọc ra từ các vấn đề nêu trong chương này. Đó là một ví dụ của điều mà `research-rules.md` mục 7 nói: câu hỏi nghiên cứu tốt mọc ra từ một vấn đề thật, không từ cảm hứng ngẫu nhiên.

Nối ngang: `../marker/` là cây bàn về một giải pháp thực dụng cho bài toán định vị tuyệt đối trong môi trường có thể sửa đổi, và nó có một phần riêng về ngân sách sai số và nghiệm thu — rất gần tinh thần chương này, ở một quy mô nhỏ hơn và cụ thể hơn.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu bốn quyết định không sửa được sau khi chốt cơ khí, và nói mỗi cái được quyết bởi phân tích nào.
2. Trình bày quy trình phân tích độ nhảy bốn bước, và nói vì sao chiều “nói dung sai” cũng quan trọng.
3. Nêu tỉ số quyết định việc hiệu chuẩn từng máy hay dùng chung, và nói vì sao quyết định có thể khác nhau giữa tham số nội và tham số ngoại.
4. Vì sao chuỗi tư thế của đồ gá nên được kiểm bằng phân tích observability trước khi chế tạo?
5. Nêu ba tầng logging và nói tầng nào quan trọng nhất cho việc chẩn đoán, cùng điều kiện để nó có giá trị.
6. Mô tả vòng lặp fleet analytics bốn bước, và nói ba thứ phải thiết kế từ trước để nó khởi động được.
7. Trình bày ba bước biến việc chọn cảm biến thành một bài toán có tiêu chí, và nêu kết luận cùng *điều kiện* của nó.

Nguồn

- [1] F. Pittaluga, S. J. Koppal, S. B. Kang, et al., “Revealing Scenes by Inverting Structure from Motion Reconstructions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của mỗi lo map inversion, 2019.
- [2] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của khung hiệu chuẩn liên tục theo thời gian bằng B-spline, 2013.
- [3] T. Schneider, M. Dymczyk, M. Fehr, et al., “maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [4] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.
- [5] M. Grupp, *evo: Python Package for the Evaluation of Odometry and SLAM*, phần mềm mã nguồn mở, mức tin C; công cụ, không phải công trình phản biện, 2017.
- [6] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của quy ước căn chỉnh và drift theo đoạn, 2018.